

Τεχνολογίες και Προγραμματισμός Εφαρμογών στον Ιστό

Εργασίες 1 και 2 (ομαδικές – 2 ατόμων)

Διδάσκουσα: Δρ. Μαριέττου Σαβίνα

Χειμερινό Εξάμηνο 2026 – 2027

Γενικές πληροφορίες

Βαθμολογία

- Η **Εργασία 1** είναι **υποχρεωτική** και η **Εργασία 2** είναι **προαιρετική**.
- Αν παραδώσετε μόνο την Εργασία 1: **Τ.Β. = 0,15 · Β(Εργασίας 1) + 0,85 · Β(γραπτού)**
- Αν παραδώσετε και τις δύο εργασίες: **Τ.Β. = 0,15 · Β(Εργασίας 1) + 0,15 · Β(Εργασίας 2) + 0,70 · Β(γραπτού)**
- Κάθε εργασία βαθμολογείται με άριστα το 10. Με το bonus ο βαθμός μιας εργασίας δεν ξεπερνά το 10.

Προφορική εξέταση

Και οι δύο εργασίες εξετάζονται προφορικά, και η εξέταση είναι υποχρεωτική. Εργασία που δεν εξετάζεται δεν βαθμολογείται.

- Εξετάζεται **κάθε μέλος της ομάδας ξεχωριστά**, οπότε ο βαθμός μπορεί να διαφέρει μεταξύ των μελών.
- Θα σας ζητηθεί να εξηγήσετε σημεία του κώδικά σας και να κάνετε μια μικρή αλλαγή ζωντανά.
- Η ημερομηνία και η σειρά εξέτασης θα ανακοινωθούν στο eclass.

Τρόπος εκπόνησης και προθεσμίες

- Ομαδική εργασία, ομάδες των **2 ατόμων**. Η ίδια ομάδα και για τις δύο εργασίες.
- Η Εργασία 1 μπορεί να υλοποιηθεί με ένα από δύο θέματα (**Θέμα Α**: Διαχείριση χρηστών, **Θέμα Β**: Mini e-shop).

  Θέμα Α: Διαχείριση χρηστών, για όποια πλατφόρμα θέλετε (π.χ. φοιτητές σε πλατφόρμα τύπου e-class, μέλη βιβλιοθήκης, μέλη κινηματογραφικού club). Αν σκοπεύετε να κάνετε και την Εργασία 2, προτείνεται το ***Θέμα Β***, γιατί είναι επέκτασή του. Λεπτομέρειες στην Εργασία 1.
- Παράδοση Εργασίας 1: **Δευτέρα 21 Δεκεμβρίου 2026, 23:59**
- Παράδοση Εργασίας 2: **Παρασκευή 15 Ιανουαρίου 2027, 23:59**

Τεχνολογίες

- Η **Εργασία 1** υλοποιείται αποκλειστικά με HTML, CSS και JavaScript, χωρίς frameworks. Λειτουργεί τοπικά με απλό άνοιγμα του `index.html`.
- Η **Εργασία 2** υλοποιείται με **React** (Vite, React Router) στον client, **Node.js με Express** στον server και **MongoDB** (Mongoose) για τη βάση δεδομένων. Εκτελείται τοπικά με `npm`.
- Έμφαση σε: καθαρό κώδικα, σωστή δομή, καλή εμπειρία χρήστη (UX) και σχολιασμό όπου χρειάζεται.

Ακαδημαϊκή ακεραιότητα

- Εργασίες με ουσιαστικά ίδιο κώδικα μεταξύ ομάδων μηδενίζονται.
- Πρέπει να μπορείτε να εξηγήσετε κάθε γραμμή του κώδικά σας. **Κώδικας που δεν μπορείτε να εξηγήσετε δεν βαθμολογείται.** Η προφορική εξέταση ελέγχει ακριβώς αυτό.

Υποβολή

- Κάθε εργασία υποβάλλεται **χωριστά**, στο **eclass** του μαθήματος, από ένα μέλος της ομάδας.
- Εκπρόθεσμες υποβολές δεν γίνονται δεκτές.
- Για κάθε εργασία, ανεβάζετε ένα συμπιεσμένο αρχείο, με όνομα τους αριθμούς μητρώου των μελών: `p3xxxxxx-p3yyyyyy.zip`. Στην Εργασία 2 το zip **δεν** περιέχει τον φάκελο `node_modules`.
- Κάθε zip περιέχει README με τα ονόματα και τους ΑΜ των μελών, οδηγίες εκτέλεσης και σύντομη περιγραφή.

Ενδεικτικά εργαλεία

- **Visual Studio Code** για τη συγγραφή κώδικα.
- **Developer Tools** του browser (Chrome, Firefox, Edge) για έλεγχο DOM, CSS, κονσόλας και αιτημάτων δικτύου.
- Για την Εργασία 1 αρκεί ένας browser: δεν χρειάζεται server ή εγκατάσταση.
- Για την Εργασία 2: **Node.js** (μαζί εγκαθίσταται και το npm) και **MongoDB Community Server** με **MongoDB Compass.** Προαιρετικά, **Postman** για δοκιμές του API.

Εργασία 1: Εφαρμογή Διαχείρισης Δεδομένων (CRUD)

Σκοπός: Υλοποίηση εφαρμογής μίας σελίδας (Single-Page) για τη διαχείριση δεδομένων (CRUD), χωρίς ανανέωση της σελίδας.

Επιλογή θέματος

Η εργασία μπορεί να υλοποιηθεί με ένα από τα δύο παρακάτω θέματα. Οι τεχνικές απαιτήσεις και η βαθμολόγηση είναι ίδιες και για τα δύο.

- **Θέμα Α: Διαχείριση χρηστών.** Υλοποιείτε την εκφώνηση ακριβώς όπως περιγράφεται παρακάτω.
- **Θέμα Β: Mini e-shop.** Προτείνεται αν σκοπεύετε να κάνετε και την **Εργασία 2**. Ισχύουν οι ίδιες απαιτήσεις, αλλά για προϊόντα αντί για χρήστες: πίνακας προϊόντων (όνομα, κατηγορία, τιμή, διαθεσιμότητα) με προσθήκη, επεξεργασία, διαγραφή, ταξινόμηση, αναζήτηση και φίλτρο κατηγορίας, κλάσεις Product και ProductStore, αποθήκευση στο κλειδί products. Η φόρμα εγγραφής με το live validation παραμένει ίδια, ως φόρμα εγγραφής πελάτη.

Απαιτήσεις

1. Φόρμα εισαγωγής χρήστη

Πεδία: Όνομα, Επώνυμο, Email, Ρόλος (Admin/User), Ημερομηνία εγγραφής, Password, Επιβεβαίωση password. Όλα τα πεδία είναι υποχρεωτικά. Η ημερομηνία εγγραφής προσυμπληρώνεται με τη σημερινή ημερομηνία.

2. Live validation

Σε κάθε πεδίο εμφανίζεται άμεσα «Έγκυρο» ή σχετικό μήνυμα λάθους.

- Το Email πρέπει να έχει έγκυρη μορφή και να είναι μοναδικό (δεν επιτρέπονται διπλότυπα).
- Όνομα και Επώνυμο: κεφαλαία ελληνικά χωρίς τόνους. Επιτρέπεται κενό ή παύλα για σύνθετα ονόματα. Έγκυρο: ΓΕΩΡΓΙΟΣ, ΑΝΝΑ-ΜΑΡΙΑ. Μη έγκυρο: Γεώργιος (πεζά), ΓΙΩΡΓΟΣ (τόνος), GEORGE (λατινικοί χαρακτήρες).
- Το Password πρέπει να έχει τουλάχιστον 8 χαρακτήρες και να περιλαμβάνει τουλάχιστον έναν αριθμό, ένα γράμμα και ένα από τα σύμβολα `!@#$%^&*`. Έγκυρο: 123A$6789. Μη έγκυρο: Abcdefg1 (λείπει σύμβολο), 12345678$ (λείπει γράμμα), Ab1$ (πολύ σύντομο).
- Η επιβεβαίωση του password πρέπει να ταιριάζει με το password.
- Η επικύρωση γίνεται με HTML γνωρίσματα (`required`, `minlength`, `pattern`) **και** με το Constraint Validation API (`setCustomValidity`, `checkValidity`).
- Δίπλα στο password εμφανίζεται ένδειξη (✓ / ✗) για κάθε κανόνα πολυπλοκότητας.

3. Πίνακας χρηστών

Εμφάνιση όλων των χρηστών με δυνατότητα **Επεξεργασίας** και **Διαγραφής** για κάθε γραμμή. Πριν από τη διαγραφή ζητείται επιβεβαίωση.

4. Ταξινόμηση

Ταξινόμηση στηλών (όνομα, email, ημερομηνία), αύξουσα και φθίνουσα.

5. Αναζήτηση και φίλτρο

- Πεδίο αναζήτησης με λέξη-κλειδί σε όνομα, επώνυμο ή email, που φιλτράρει τον πίνακα καθώς πληκτρολογείται.
- Φίλτρο ρόλου (Όλοι / Admin / User).
- Μήνυμα «Δεν βρέθηκαν χρήστες» όταν δεν υπάρχουν αποτελέσματα.

6. Δομή κώδικα και αποθήκευση

- Κλάση `User` για τα στοιχεία ενός χρήστη.
- Ξεχωριστή κλάση για τη διαχείριση των χρηστών (π.χ. `UserStore`), με μεθόδους προσθήκης, αναζήτησης, ενημέρωσης και διαγραφής.
- Οι χρήστες κρατιούνται σε πίνακα JavaScript και αποθηκεύονται μόνιμα στο `localStorage`, ώστε να παραμένουν μετά από ανανέωση της σελίδας ή κλείσιμο του browser.
- Ο πίνακας αποθηκεύεται ως JSON (`JSON.stringify`) σε ένα κλειδί με όνομα `users`, και διαβάζεται ξανά με `JSON.parse` κατά τη φόρτωση της σελίδας. Η αποθήκευση ενημερώνεται μετά από κάθε προσθήκη, επεξεργασία και διαγραφή.
- **Έλεγχος:** τα αποθηκευμένα δεδομένα φαίνονται στα Developer Tools του browser (F12 → Application → Local Storage → η σελίδα σας → κλειδί `users`). Προσθέστε έναν χρήστη, ανανεώστε τη σελίδα, και ο χρήστης πρέπει να εμφανίζεται ακόμα στον πίνακα.
- Σημείωση: σε πραγματική εφαρμογή οι κωδικοί δεν αποθηκεύονται ποτέ στον browser. Εδώ επιτρέπεται μόνο για λόγους εξάσκησης.

7. Εμφάνιση

- Σημασιολογικά στοιχεία HTML5 (`header`, `main`, `section`, `form`, `table`, `footer`) και σωστά `label` σε όλα τα πεδία.
- Responsive διάταξη: η σελίδα είναι χρηστική και σε κινητό, χωρίς οριζόντια κύλιση ολόκληρης της σελίδας.

8. Bonus (έως +10%)

Μπορείτε να υλοποιήσετε μία ή περισσότερες από τις παρακάτω επεκτάσεις. Κάθε μία δίνει έως +5%, με μέγιστο συνολικό bonus +10%.

- **Εξαγωγή και εισαγωγή:** κουμπί που κατεβάζει τους χρήστες σε αρχείο CSV ή JSON, και δυνατότητα φόρτωσης χρηστών από αρχείο JSON, με έλεγχο εγκυρότητας κάθε εγγραφής (με τους ίδιους κανόνες της φόρμας) και απόρριψη διπλότυπων email.
- **Σελιδοποίηση:** εμφάνιση του πίνακα ανά σελίδες (π.χ. 10 χρήστες ανά σελίδα), που λειτουργεί σωστά μαζί με την αναζήτηση, το φίλτρο και την ταξινόμηση.
- **Αναίρεση διαγραφής:** μετά από διαγραφή εμφανίζεται μήνυμα με κουμπί «Αναίρεση» για λίγα δευτερόλεπτα, που επαναφέρει τον χρήστη στη θέση του.

Παραδοτέα

- Πηγαίος κώδικας: `index.html`, `style.css`, `logic.js`.
- README με τα στοιχεία της ομάδας, οδηγίες εκτέλεσης και σύντομη περιγραφή.
- Screenshots σε ένα αρχείο screenshots.pdf, με μία σύντομη λεζάντα κάτω από κάθε εικόνα: φόρμα με validation (έγκυρα και μη έγκυρα πεδία), πίνακας χρηστών, ταξινόμηση, αναζήτηση/φίλτρο, επεξεργασία, προβολή σε κινητό, και το κλειδί users στο Local Storage των Developer Tools.

Βαθμολόγηση Εργασίας 1

- Φόρμα και live validation: **20%**
- CRUD (προσθήκη, επεξεργασία, διαγραφή, πίνακας): **20%**
- Ταξινόμηση στηλών: **10%**
- Αναζήτηση και φίλτρο ρόλου: **10%**
- Password με κανόνες και Constraint Validation API: **10%**
- Σημασιολογικό HTML, CSS και responsive διάταξη: **10%**
- Σχεδίαση κώδικα και αποθήκευση (κλάσεις `User` / `UserStore`, `localStorage`, σχόλια): **10%**
- Εμπειρία χρήστη (ευχρηστία, μηνύματα, ροή): **5%**
- README και screenshots: **5%**
- **Bonus:** εξαγωγή/εισαγωγή, σελιδοποίηση, αναίρεση διαγραφής: **έως +10%**

Εργασία 2: Mini E-shop (Κατάλογος, Καλάθι, Ταυτοποίηση)

Σκοπός: Υλοποίηση ολοκληρωμένης διαδικτυακής εφαρμογής ηλεκτρονικού καταστήματος με κατάλογο προϊόντων, καλάθι αγορών, ολοκλήρωση παραγγελίας και σύνδεση χρηστών με ρόλους.

Σχέση με την Εργασία 1: Αν υλοποιήσατε την Εργασία 1 με το Θέμα Β (mini e-shop), η Εργασία 2 είναι η επέκτασή της. Ξαναχρησιμοποιήστε τα δεδομένα, τη λογική επικύρωσης και την CSS σας: τα προϊόντα και οι χρήστες μεταφέρονται από το `localStorage` στη MongoDB, και η σελίδα ξαναγράφεται σε React. Αν υλοποιήσατε το Θέμα Α, ξεκινάτε νέο έργο, αλλά μπορείτε να ξαναχρησιμοποιήσετε τη λογική επικύρωσης της φόρμας.

Αρχιτεκτονική

Η εφαρμογή αποτελείται από δύο ξεχωριστά έργα, `client/` και `server/`:

- **Πελάτης (client):** εφαρμογή σε **React**, που δημιουργείται με **Vite**. Η επικοινωνία με τον server γίνεται με `fetch` και αιτήματα JSON. Στο `vite.config.js` ορίζεται proxy, ώστε τα αιτήματα προς `/api` να προωθούνται στον server.
- **Διακομιστής (server): Node.js** με **Express**, που παρέχει REST API. Όλα τα endpoints επιστρέφουν JSON.
- **Βάση δεδομένων: MongoDB**, με χρήση **Mongoose**.

Απαιτήσεις

1. Βάση δεδομένων

- Τα collections `users`, `products` και `orders`, με σχήματα Mongoose (π.χ. υποχρεωτικά πεδία, μοναδικό email).
- Κάθε παραγγελία αποθηκεύεται ως ένα έγγραφο, που περιέχει τον χρήστη, την ημερομηνία, το σύνολο και τα προϊόντα της (όνομα, τιμή, ποσότητα).
- Script αρχικοποίησης (`npm run seed`) που γεμίζει τη βάση με τουλάχιστον 10 προϊόντα, έναν Admin και έναν User.

2. Κατάλογος προϊόντων

- Τα προϊόντα (όνομα, κατηγορία, τιμή, διαθεσιμότητα) φορτώνονται από τον server με `GET /api/products`.
- Αναζήτηση βάσει ονόματος, φίλτρο κατηγορίας και ταξινόμηση βάσει τιμής. Μπορούν να γίνονται στη React.

3. Καλάθι αγορών και παραγγελία

- Προσθήκη/αφαίρεση προϊόντων, αλλαγή ποσότητας και υπολογισμός συνόλου. Το καλάθι διατηρείται μετά από ανανέωση της σελίδας (`localStorage`).
- Ολοκλήρωση παραγγελίας: ο client στέλνει **μόνο** τα id των προϊόντων και τις ποσότητες. Ο server **βρίσκει τις τιμές από τη βάση**, υπολογίζει το σύνολο και αποθηκεύει την παραγγελία. Ο server δεν εμπιστεύεται τιμές που στέλνει ο client.
- Ο User βλέπει το ιστορικό των δικών του παραγγελιών.

4. Εγγραφή και σύνδεση (Authentication)

- Εγγραφή, σύνδεση και αποσύνδεση. Οι κανόνες της φόρμας εγγραφής είναι ίδιοι με την Εργασία 1.
- Οι κωδικοί αποθηκεύονται ως hash με **bcrypt**, ποτέ σε απλό κείμενο, και δεν επιστρέφονται ποτέ από το API.
- Η σύνδεση γίνεται με **JWT**: μετά από επιτυχές login ο server επιστρέφει ένα token, και ο client το στέλνει σε κάθε επόμενο αίτημα στο header `Authorization: Bearer <token>`.

5. Ρόλοι (Authorization)

- **User:** βλέπει τον κατάλογο, χρησιμοποιεί το καλάθι, κάνει παραγγελίες και βλέπει μόνο τις δικές του.
- **Admin:** προσθέτει, επεξεργάζεται και διαγράφει προϊόντα.
- Ο έλεγχος δικαιωμάτων γίνεται **στον server**, με middleware στο Express. Η απόκρυψη κουμπιών στη React δεν αρκεί. Αίτημα χωρίς σύνδεση απορρίπτεται με 401 και αίτημα χωρίς δικαίωμα με 403.

6. Validation και ασφάλεια

- Έλεγχος δεδομένων **και στον client και στον server**. Τα μη έγκυρα δεδομένα (π.χ. αρνητική ποσότητα, κενό όνομα) απορρίπτονται στον server με 400 και μήνυμα JSON.
- Τα μυστικά (σύνδεση με τη βάση, κλειδί του JWT) βρίσκονται σε αρχείο `.env`, που **δεν** παραδίδεται. Παραδίδεται μόνο `.env.example`.

7. REST API

- Σωστή χρήση των HTTP methods (GET, POST, PUT, DELETE) και των κωδικών 200, 201, 400, 401, 403, 404. Ενδεικτικά endpoints:

```
POST /api/auth/register, POST /api/auth/login
GET /api/products
POST /api/products, PUT /api/products/:id, DELETE /api/products/:id
(Admin)
GET /api/orders, POST /api/orders
```

8. Εφαρμογή React

- Components (π.χ. `ProductCard`, `Navbar`, `Cart`) και τουλάχιστον οι σελίδες: κατάλογος, καλάθι, σύνδεση/εγγραφή, οι παραγγελίες μου, διαχείριση προϊόντων (Admin).
- Χρήση `useState` και `useEffect`. Οι κλήσεις προς το API συγκεντρώνονται σε ένα αρχείο (π.χ. `api.js`).
- Σαφή μηνύματα επιτυχίας και σφάλματος για τον χρήστη.

Παραδοτέα

- Οι φάκελοι `client/` και `server/`, **χωρίς** `node_modules` και **χωρίς** `.env`.
- README με τα στοιχεία της ομάδας, τα βήματα εκτέλεσης (`npm install`, ρύθμιση `.env`, `npm run seed`, `npm run dev`) και τους λογαριασμούς δοκιμής (Admin/User).
- Screenshots σε ένα αρχείο screenshots.pdf, με μία σύντομη λεζάντα κάτω από κάθε εικόνα: κατάλογος, καλάθι, παραγγελία, σύνδεση, διαχείριση προϊόντων.
- **Πίνακας δοκιμών** με τουλάχιστον 5 περιπτώσεις και το αποτέλεσμα κάθε μίας, π.χ.: λάθος κωδικός, διπλό email, User που προσπαθεί να διαγράψει προϊόν, αρνητική ποσότητα, τροποποιημένη τιμή από τον client.

Βαθμολόγηση Εργασίας 2

- Βάση δεδομένων (σχήματα, seed): **10%**
- Κατάλογος (φόρτωση από τον server, αναζήτηση, φίλτρο, ταξινόμηση): **10%**
- Καλάθι και παραγγελία (τιμές από τη βάση, ιστορικό): **15%**
- Εγγραφή και σύνδεση (bcrypt, JWT): **15%**
- Ρόλοι (έλεγχος στον server): **15%**
- Validation και ασφάλεια (client και server, .env): **10%**
- REST API (methods, κωδικοί, δομή server): **10%**
- Εφαρμογή React (components, σελίδες, api.js): **10%**
- README, screenshots, πίνακας δοκιμών: **5%**

Bonus (έως +10%)

Κάθε επέκταση δίνει έως +5%, με μέγιστο συνολικό bonus +10%.

- **Αξιολογήσεις προϊόντων:** ο User βαθμολογεί (1–5) και σχολιάζει ένα προϊόν, και ο μέσος όρος εμφανίζεται στον κατάλογο.
- **Κατάσταση παραγγελίας:** ο Admin βλέπει όλες τις παραγγελίες και αλλάζει την κατάστασή τους (π.χ. Σε επεξεργασία → Απεστάλη), και ο User βλέπει την ενημερωμένη κατάσταση.
- **Σελιδοποίηση:** ο server επιστρέφει τα προϊόντα ανά σελίδες (π.χ. `GET /api/products?page=2`), με κουμπιά σελίδων στη React.